Assignment 2.1

Mostafa Zamaniturk

Background

You work as a data scientist at an advertising agency. Your task is to analyze the advertising dataset to predict whether a user will click on an online ad based on several factors. The dataset contains information about whether a user clicked on an online ad as well as the following factors:

Daily time spent on site

Age

Area income

Daily internet usage

Ad topic line

City

Male

Country

Timestamp

Clicked on ad

Objective

Your task is to build an ANN classification model that can predict whether a user will click on an online ad based on the given factors.

Dataset

Advertising Dataset

Instructions

1. Load the dataset into pandas dataframe.
2. Perform data cleaning and preprocessing as necessary.
3. Split the data into training and testing sets using an 80:20 ratio.
4. Scale the data using StandardScaler.
5. Build an ANN classification model.
6. Experiment with different model architectures, activation functions, regularization techniques, learning rates, and batch sizes to optimize the model's performance.
7. Evaluate the model's performance using accuracy, precision, recall, F1 score, and ROC AUC score as the metrics.
8. Interpret the results and draw conclusions about the factors that are most important in predicting whether a user will click on an online ad.

In [1]:
```
# 1. Load the dataset into pandas dataframe.

import zipfile
from pathlib import Path

import pandas as pd

zip_path = Path("M2-Advertising Dataset.zip")
extract_dir = Path(".")

with zipfile.ZipFile(zip_path, "r") as zf:
    zf.extractall(extract_dir)

df = pd.read_csv(extract_dir / "M2-Advertising Dataset.csv")
df.head()
```

Out[1]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Ad Topic Line	City	Male	Country	Times
0	68.95	35	61833.90	256.09	Cloned 5thgeneration orchestration	Wrightburgh	0	Tunisia	2016 27 00:
1	80.23	31	68441.85	193.77	Monitored national standardization	West Jodi	1	Nauru	2016 01::
2	69.47	26	59785.94	236.50	Organic bottom-line service-desk	Davidton	0	San Marino	2016 20::
3	74.15	29	54806.18	245.89	Triple-buffered reciprocal time-frame	West Terrifurt	1	Italy	2016-02:
4	68.37	35	73889.99	225.58	Robust logistical utilization	South Manuel	0	Iceland	2016 03:

In [3]:

```
# 2. Perform data cleaning and preprocessing as necessary.

# Inspect data quality
print("Missing values:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
print("\nTarget balance:\n", df["Clicked on Ad"].value_counts())

# Work on a copy so the raw dataframe stays unchanged
data = df.copy()

# Separate target variable
y = data["Clicked on Ad"]
data = data.drop(columns=["Clicked on Ad"])

# Drop high-cardinality text columns that act like identifiers and hurt gene
# Ad Topic Line is unique for every row; City is nearly unique
data = data.drop(columns=["Ad Topic Line", "City"])

# Extract useful numeric features from Timestamp
data["Timestamp"] = pd.to_datetime(data["Timestamp"])
data["Hour"] = data["Timestamp"].dt.hour
data["DayOfWeek"] = data["Timestamp"].dt.dayofweek
data["Month"] = data["Timestamp"].dt.month
data = data.drop(columns=["Timestamp"])

# One-hot encode Country (nominal categorical feature)
data = pd.get_dummies(data, columns=["Country"], drop_first=True)

# Final feature matrix (all numeric for scaling and ANN training)
X = data.astype(float)
```

```
print("\nPreprocessed feature shape:", X.shape)
print("Target shape:", y.shape)
print("\nFeature columns:")
print(X.columns.tolist())

X.head()
```

```
Missing values:
 Daily Time Spent on Site    0
Age                          0
Area Income                  0
Daily Internet Usage         0
Ad Topic Line                0
City                         0
Male                         0
Country                      0
Timestamp                    0
Clicked on Ad                0
dtype: int64

Duplicate rows: 0

Target balance:
 Clicked on Ad
0    500
1    500
Name: count, dtype: int64

Preprocessed feature shape: (1000, 244)
Target shape: (1000,)

Feature columns:
['Daily Time Spent on Site', 'Age', 'Area Income', 'Daily Internet Usage',
'Male', 'Hour', 'DayOfWeek', 'Month', 'Country_Albania', 'Country_Algeria',
'Country_American Samoa', 'Country_Andorra', 'Country_Angola', 'Country_Angu
illa', 'Country_Antarctica (the territory South of 60 deg S)', 'Country_Anti
gua and Barbuda', 'Country_Argentina', 'Country_Armenia', 'Country_Aruba',
'Country_Australia', 'Country_Austria', 'Country_Azerbaijan', 'Country_Baham
as', 'Country_Bahrain', 'Country_Bangladesh', 'Country_Barbados', 'Country_B
elarus', 'Country_Belgium', 'Country_Belize', 'Country_Benin', 'Country_Berm
uda', 'Country_Bhutan', 'Country_Bolivia', 'Country_Bosnia and Herzegovina',
'Country_Bouvet Island (Bouvetoya)', 'Country_Brazil', 'Country_British Indi
an Ocean Territory (Chagos Archipelago)', 'Country_British Virgin Islands',
'Country_Brunei Darussalam', 'Country_Bulgaria', 'Country_Burkina Faso', 'Co
untry_Burundi', 'Country_Cambodia', 'Country_Cameroon', 'Country_Canada', 'C
ountry_Cape Verde', 'Country_Cayman Islands', 'Country_Central African Repub
lic', 'Country_Chad', 'Country_Chile', 'Country_China', 'Country_Christmas I
sland', 'Country_Colombia', 'Country_Comoros', 'Country_Congo', 'Country_Coo
k Islands', 'Country_Costa Rica', "Country_Cote d'Ivoire", 'Country_Croati
a', 'Country_Cuba', 'Country_Cyprus', 'Country_Czech Republic', 'Country_Den
mark', 'Country_Djibouti', 'Country_Dominica', 'Country_Dominican Republic',
'Country_Ecuador', 'Country_Egypt', 'Country_El Salvador', 'Country_Equatori
al Guinea', 'Country_Eritrea', 'Country_Estonia', 'Country_Ethiopia', 'Count
ry_Falkland Islands (Malvinas)', 'Country_Faroe Islands', 'Country_Fiji', 'C
ountry_Finland', 'Country_France', 'Country_French Guiana', 'Country_French
Polynesia', 'Country_French Southern Territories', 'Country_Gabon', 'Country
_Gambia', 'Country_Georgia', 'Country_Germany', 'Country_Ghana', 'Country_Gi
braltar', 'Country_Greece', 'Country_Greenland', 'Country_Grenada', 'Country
_Guadeloupe', 'Country_Guam', 'Country_Guatemala', 'Country_Guernsey', 'Coun
try_Guinea', 'Country_Guinea-Bissau', 'Country_Guyana', 'Country_Haiti', 'Co
untry_Heard Island and McDonald Islands', 'Country_Holy See (Vatican City St
ate)', 'Country_Honduras', 'Country_Hong Kong', 'Country_Hungary', 'Country_
Iceland', 'Country_India', 'Country_Indonesia', 'Country_Iran', 'Country_Ire
```

```
land', 'Country_Isle of Man', 'Country_Israel', 'Country_Italy', 'Country_Ja
maica', 'Country_Japan', 'Country_Jersey', 'Country_Jordan', 'Country_Kazakh
stan', 'Country_Kenya', 'Country_Kiribati', 'Country_Korea', 'Country_Kuwai
t', 'Country_Kyrgyz Republic', "Country_Lao People's Democratic Republic",
'Country_Latvia', 'Country_Lebanon', 'Country_Lesotho', 'Country_Liberia',
'Country_Libyan Arab Jamahiriya', 'Country_Liechtenstein', 'Country_Lithuani
a', 'Country_Luxembourg', 'Country_Macao', 'Country_Macedonia', 'Country_Mad
agascar', 'Country_Malawi', 'Country_Malaysia', 'Country_Maldives', 'Country
_Mali', 'Country_Malta', 'Country_Marshall Islands', 'Country_Martinique',
'Country_Mauritania', 'Country_Mauritius', 'Country_Mayotte', 'Country_Mexic
o', 'Country_Micronesia', 'Country_Moldova', 'Country_Monaco', 'Country_Mong
olia', 'Country_Montenegro', 'Country_Montserrat', 'Country_Morocco', 'Count
ry_Mozambique', 'Country_Myanmar', 'Country_Namibia', 'Country_Nauru', 'Coun
try_Nepal', 'Country_Netherlands', 'Country_Netherlands Antilles', 'Country_
New Caledonia', 'Country_New Zealand', 'Country_Nicaragua', 'Country_Niger',
'Country_Niue', 'Country_Norfolk Island', 'Country_Northern Mariana Island
s', 'Country_Norway', 'Country_Pakistan', 'Country_Palau', 'Country_Palestin
ian Territory', 'Country_Panama', 'Country_Papua New Guinea', 'Country_Parag
uay', 'Country_Peru', 'Country_Philippines', 'Country_Pitcairn Islands', 'Co
untry_Poland', 'Country_Portugal', 'Country_Puerto Rico', 'Country_Qatar',
'Country_Reunion', 'Country_Romania', 'Country_Russian Federation', 'Country
_Rwanda', 'Country_Saint Barthelemy', 'Country_Saint Helena', 'Country_Saint
Kitts and Nevis', 'Country_Saint Lucia', 'Country_Saint Martin', 'Country_Sa
int Pierre and Miquelon', 'Country_Saint Vincent and the Grenadines', 'Count
ry_Samoa', 'Country_San Marino', 'Country_Sao Tome and Principe', 'Country_S
audi Arabia', 'Country_Senegal', 'Country_Serbia', 'Country_Seychelles', 'Co
untry_Sierra Leone', 'Country_Singapore', 'Country_Slovakia (Slovak Republi
c)', 'Country_Slovenia', 'Country_Somalia', 'Country_South Africa', 'Country
_South Georgia and the South Sandwich Islands', 'Country_Spain', 'Country_Sr
i Lanka', 'Country_Sudan', 'Country_Suriname', 'Country_Svalbard & Jan Mayen
Islands', 'Country_Swaziland', 'Country_Sweden', 'Country_Switzerland', 'Cou
ntry_Syrian Arab Republic', 'Country_Taiwan', 'Country_Tajikistan', 'Country
_Tanzania', 'Country_Thailand', 'Country_Timor-Leste', 'Country_Togo', 'Coun
try_Tokelau', 'Country_Tonga', 'Country_Trinidad and Tobago', 'Country_Tunis
ia', 'Country_Turkey', 'Country_Turkmenistan', 'Country_Turks and Caicos Isl
ands', 'Country_Tuvalu', 'Country_Uganda', 'Country_Ukraine', 'Country_Unite
d Arab Emirates', 'Country_United Kingdom', 'Country_United States Minor Out
lying Islands', 'Country_United States Virgin Islands', 'Country_United Stat
es of America', 'Country_Uruguay', 'Country_Uzbekistan', 'Country_Vanuatu',
'Country_Venezuela', 'Country_Vietnam', 'Country_Wallis and Futuna', 'Countr
y_Western Sahara', 'Country_Yemen', 'Country_Zambia', 'Country_Zimbabwe']
```

Out[3]:

	Daily Time Spent on Site	Age	Area Income	Daily Internet Usage	Male	Hour	DayOfWeek	Month	Country_Albania
0	68.95	35.0	61833.90	256.09	0.0	0.0	6.0	3.0	0.0
1	80.23	31.0	68441.85	193.77	1.0	1.0	0.0	4.0	0.0
2	69.47	26.0	59785.94	236.50	0.0	20.0	6.0	3.0	0.0
3	74.15	29.0	54806.18	245.89	1.0	2.0	6.0	1.0	0.0
4	68.37	35.0	73889.99	225.58	0.0	3.0	4.0	6.0	0.0

5 rows × 244 columns

In [6]:
```
# 3. Split the data into training and testing sets using an 80:20 ratio.

import tensorflow as tf

tf.random.set_seed(42)

num_samples = X.shape[0]
train_size = int(0.8 * num_samples)
test_size = num_samples - train_size

# Convert preprocessed data to TensorFlow tensors
X_tf = tf.constant(X.values, dtype=tf.float32)
y_tf = tf.constant(y.values, dtype=tf.float32)

# Shuffle indices and split 80/20
indices = tf.random.shuffle(tf.range(num_samples))
train_indices = indices[:train_size]
test_indices = indices[train_size:]

X_train = tf.gather(X_tf, train_indices)
X_test = tf.gather(X_tf, test_indices)
y_train = tf.gather(y_tf, train_indices)
y_test = tf.gather(y_tf, test_indices)

print(f"Training samples: {X_train.shape[0]} ({train_size / num_samples:.0%}
print(f"Testing samples: {X_test.shape[0]} ({test_size / num_samples:.0%})")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train class balance: {tf.reduce_sum(y_train)} clicked, {len(y_trai
print(f"y_test class balance: {tf.reduce_sum(y_test)} clicked, {len(y_test)
```

```
Training samples: 800 (80%)
Testing samples: 200 (20%)
X_train shape: (800, 244)
X_test shape: (200, 244)
y_train class balance: 404.0 clicked, 396 not clicked
y_test class balance: 96.0 clicked, 104 not clicked
```

In [7]:
```python
# 4. Scale the data using StandardScaler.

from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print("Scaled training set stats (first 5 features):")
print("Means:", X_train[:, :5].mean(axis=0))
print("Stds:", X_train[:, :5].std(axis=0))
```

```
X_train shape: (800, 244)
X_test shape: (200, 244)
Scaled training set stats (first 5 features):
Means: [ 4.17027524e-16 -1.89015470e-16 -2.96984659e-17  1.44606549e-16
  2.47024623e-17]
Stds: [1. 1. 1. 1. 1.]
```

In [8]:
```python
# 5. Build an ANN classification model.

from tensorflow import keras
from tensorflow.keras import layers

tf.random.set_seed(42)

input_dim = X_train.shape[1]

model = keras.Sequential(
    [
        layers.Input(shape=(input_dim,)),
        layers.Dense(128, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid"),
    ],
    name="ann_click_classifier",
)

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"],
)

model.summary()

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
```

```
    epochs=50,
    batch_size=32,
    verbose=1,
)
```

Model: "ann_click_classifier"

Layer (type)	Output Shape	Par
dense (Dense)	(None, 128)	31
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 64)	8
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 32)	2
dropout_2 (Dropout)	(None, 32)	
dense_3 (Dense)	(None, 1)	

Total params: 41,729 (163.00 KB)
Trainable params: 41,729 (163.00 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.5281 - loss: 0.7149 - v
al_accuracy: 0.6562 - val_loss: 0.6423
Epoch 2/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6250 - loss: 0.6501 - v
al_accuracy: 0.6812 - val_loss: 0.6025
Epoch 3/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.7297 - loss: 0.5424 - v
al_accuracy: 0.7500 - val_loss: 0.5252
Epoch 4/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.8031 - loss: 0.4446 - v
al_accuracy: 0.8500 - val_loss: 0.4148
Epoch 5/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8594 - loss: 0.3216 - v
al_accuracy: 0.8625 - val_loss: 0.3105
Epoch 6/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9141 - loss: 0.2125 - v
al_accuracy: 0.8687 - val_loss: 0.2786
Epoch 7/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9500 - loss: 0.1528 - v
al_accuracy: 0.8813 - val_loss: 0.2616
Epoch 8/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9594 - loss: 0.1191 - v
al_accuracy: 0.8875 - val_loss: 0.2596
Epoch 9/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9688 - loss: 0.0881 - v
al_accuracy: 0.9000 - val_loss: 0.2571
Epoch 10/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9781 - loss: 0.0666 - v
al_accuracy: 0.9000 - val_loss: 0.2720
Epoch 11/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9797 - loss: 0.0675 - v
al_accuracy: 0.9125 - val_loss: 0.2890
Epoch 12/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9812 - loss: 0.0475 - v
al_accuracy: 0.9000 - val_loss: 0.3013
Epoch 13/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9922 - loss: 0.0292 - v
al_accuracy: 0.9187 - val_loss: 0.3102
Epoch 14/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9891 - loss: 0.0346 - v
al_accuracy: 0.9187 - val_loss: 0.3308
Epoch 15/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9828 - loss: 0.0413 - v
al_accuracy: 0.9062 - val_loss: 0.3661
Epoch 16/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9750 - loss: 0.0541 - v
al_accuracy: 0.9187 - val_loss: 0.3359
Epoch 17/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9937 - loss: 0.0223 - v
al_accuracy: 0.9187 - val_loss: 0.3528
Epoch 18/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9953 - loss: 0.0173 - v
al_accuracy: 0.9187 - val_loss: 0.3558
Epoch 19/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9891 - loss: 0.0308 - v
```

```
al_accuracy: 0.9187 - val_loss: 0.3538
Epoch 20/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9875 - loss: 0.0350 - v
al_accuracy: 0.9250 - val_loss: 0.3423
Epoch 21/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0109 - v
al_accuracy: 0.9250 - val_loss: 0.3440
Epoch 22/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9969 - loss: 0.0090 - v
al_accuracy: 0.9187 - val_loss: 0.3701
Epoch 23/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 1.0000 - loss: 0.0080 - v
al_accuracy: 0.9187 - val_loss: 0.3778
Epoch 24/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9953 - loss: 0.0170 - v
al_accuracy: 0.9187 - val_loss: 0.3641
Epoch 25/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0080 - v
al_accuracy: 0.9187 - val_loss: 0.3838
Epoch 26/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9937 - loss: 0.0209 - v
al_accuracy: 0.9250 - val_loss: 0.3624
Epoch 27/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0103 - v
al_accuracy: 0.9187 - val_loss: 0.3714
Epoch 28/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0078 - v
al_accuracy: 0.9250 - val_loss: 0.3724
Epoch 29/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9953 - loss: 0.0110 - v
al_accuracy: 0.9125 - val_loss: 0.3870
Epoch 30/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0083 - v
al_accuracy: 0.9187 - val_loss: 0.4047
Epoch 31/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0116 - v
al_accuracy: 0.9250 - val_loss: 0.4006
Epoch 32/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0066 - v
al_accuracy: 0.9312 - val_loss: 0.4242
Epoch 33/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9953 - loss: 0.0137 - v
al_accuracy: 0.9187 - val_loss: 0.4412
Epoch 34/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0089 - v
al_accuracy: 0.9125 - val_loss: 0.4513
Epoch 35/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0060 - v
al_accuracy: 0.9312 - val_loss: 0.4353
Epoch 36/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0053 - v
al_accuracy: 0.9312 - val_loss: 0.4380
Epoch 37/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0049 - v
al_accuracy: 0.9312 - val_loss: 0.4339
Epoch 38/50
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 1.0000 - loss: 0.0030 - v
al_accuracy: 0.9312 - val_loss: 0.4389
Epoch 39/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 1.0000 - loss: 0.0050 - v
al_accuracy: 0.9250 - val_loss: 0.4558
Epoch 40/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 1.0000 - loss: 0.0010 - v
al_accuracy: 0.9250 - val_loss: 0.4715
Epoch 41/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0017 - v
al_accuracy: 0.9250 - val_loss: 0.4803
Epoch 42/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9984 - loss: 0.0044 - v
al_accuracy: 0.9312 - val_loss: 0.4501
Epoch 43/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0050 - v
al_accuracy: 0.9312 - val_loss: 0.4531
Epoch 44/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0043 - v
al_accuracy: 0.9312 - val_loss: 0.4825
Epoch 45/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0014 - v
al_accuracy: 0.9312 - val_loss: 0.4953
Epoch 46/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9953 - loss: 0.0169 - v
al_accuracy: 0.9312 - val_loss: 0.4731
Epoch 47/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9969 - loss: 0.0124 - v
al_accuracy: 0.9312 - val_loss: 0.4590
Epoch 48/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9937 - loss: 0.0169 - v
al_accuracy: 0.9375 - val_loss: 0.4214
Epoch 49/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 1.0000 - loss: 0.0032 - v
al_accuracy: 0.9438 - val_loss: 0.4154
Epoch 50/50
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9984 - loss: 0.0023 - v
al_accuracy: 0.9375 - val_loss: 0.4299
```

In [9]:

```python
# 6. Experiment with different model architectures, activation functions, re

from tensorflow.keras import regularizers

# Use only core numeric/time features for a smaller input dimension
core_features = [
    "Daily Time Spent on Site",
    "Age",
    "Area Income",
    "Daily Internet Usage",
    "Male",
    "Hour",
    "DayOfWeek",
    "Month",
]
core_idx = [X.columns.get_loc(col) for col in core_features]
```

```
X_train_alt = X_train[:, core_idx]
X_test_alt = X_test[:, core_idx]
input_dim_alt = X_train_alt.shape[1]

model2 = keras.Sequential(
    [
        layers.Input(shape=(input_dim_alt,)),
        layers.Dense(
            128,
            activation="tanh",
            kernel_regularizer=regularizers.l2(0.01),
        ),
        layers.Dropout(0.3),
        layers.Dense(
            64,
            activation="tanh",
            kernel_regularizer=regularizers.l2(0.01),
        ),
        layers.Dropout(0.3),
        layers.Dense(
            32,
            activation="tanh",
            kernel_regularizer=regularizers.l2(0.01),
        ),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid"),
    ],
    name="ann_click_classifier_v2",
)

model2.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0005),
    loss="mse",
    metrics=["accuracy"],
)

print(f"Model 1 input_dim: {input_dim}")
print(f"Model 2 input_dim: {input_dim_alt}")
model2.summary()

history2 = model2.fit(
    X_train_alt,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    verbose=1,
)
```

```
Model 1 input_dim: 244
Model 2 input_dim: 8
Model: "ann_click_classifier_v2"
```

Layer (type)	Output Shape	Par
dense_4 (Dense)	(None, 128)	1
dropout_3 (Dropout)	(None, 128)	
dense_5 (Dense)	(None, 64)	8
dropout_4 (Dropout)	(None, 64)	
dense_6 (Dense)	(None, 32)	2
dropout_5 (Dropout)	(None, 32)	
dense_7 (Dense)	(None, 1)	

Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - accuracy: 0.6750 - loss: 1.6110 -
val_accuracy: 0.9438 - val_loss: 1.4983
Epoch 2/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8922 - loss: 1.4622 - v
al_accuracy: 0.9312 - val_loss: 1.3880
Epoch 3/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9469 - loss: 1.3536 - v
al_accuracy: 0.9375 - val_loss: 1.3068
Epoch 4/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9547 - loss: 1.2729 - v
al_accuracy: 0.9438 - val_loss: 1.2341
Epoch 5/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9547 - loss: 1.1971 - v
al_accuracy: 0.9375 - val_loss: 1.1649
Epoch 6/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9609 - loss: 1.1260 - v
al_accuracy: 0.9375 - val_loss: 1.0987
Epoch 7/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9594 - loss: 1.0645 - v
al_accuracy: 0.9438 - val_loss: 1.0354
Epoch 8/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9609 - loss: 0.9997 - v
al_accuracy: 0.9438 - val_loss: 0.9755
Epoch 9/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9703 - loss: 0.9364 - v
al_accuracy: 0.9438 - val_loss: 0.9187
Epoch 10/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9719 - loss: 0.8813 - v
al_accuracy: 0.9375 - val_loss: 0.8651
Epoch 11/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9734 - loss: 0.8295 -
val_accuracy: 0.9375 - val_loss: 0.8146
Epoch 12/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9703 - loss: 0.7787 - v
al_accuracy: 0.9375 - val_loss: 0.7670
Epoch 13/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9672 - loss: 0.7348 - v
al_accuracy: 0.9375 - val_loss: 0.7223
Epoch 14/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9797 - loss: 0.6872 - v
al_accuracy: 0.9438 - val_loss: 0.6800
Epoch 15/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9750 - loss: 0.6456 - v
al_accuracy: 0.9438 - val_loss: 0.6402
Epoch 16/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9688 - loss: 0.6094 - v
al_accuracy: 0.9375 - val_loss: 0.6028
Epoch 17/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9766 - loss: 0.5679 - v
al_accuracy: 0.9375 - val_loss: 0.5675
Epoch 18/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9750 - loss: 0.5358 - v
al_accuracy: 0.9375 - val_loss: 0.5344
Epoch 19/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9750 - loss: 0.5035 - v
```

```
al_accuracy: 0.9375 - val_loss: 0.5033
Epoch 20/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9766 - loss: 0.4741 - v
al_accuracy: 0.9375 - val_loss: 0.4745
Epoch 21/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9766 - loss: 0.4430 - v
al_accuracy: 0.9312 - val_loss: 0.4474
Epoch 22/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9734 - loss: 0.4188 - v
al_accuracy: 0.9312 - val_loss: 0.4220
Epoch 23/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9797 - loss: 0.3924 - v
al_accuracy: 0.9375 - val_loss: 0.3983
Epoch 24/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9828 - loss: 0.3691 - v
al_accuracy: 0.9312 - val_loss: 0.3762
Epoch 25/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9719 - loss: 0.3494 - v
al_accuracy: 0.9375 - val_loss: 0.3555
Epoch 26/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9719 - loss: 0.3287 - v
al_accuracy: 0.9312 - val_loss: 0.3362
Epoch 27/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9828 - loss: 0.3104 - v
al_accuracy: 0.9312 - val_loss: 0.3181
Epoch 28/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9766 - loss: 0.2924 - v
al_accuracy: 0.9312 - val_loss: 0.3013
Epoch 29/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9719 - loss: 0.2766 - v
al_accuracy: 0.9312 - val_loss: 0.2856
Epoch 30/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9750 - loss: 0.2610 - v
al_accuracy: 0.9312 - val_loss: 0.2711
Epoch 31/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9750 - loss: 0.2469 - v
al_accuracy: 0.9375 - val_loss: 0.2574
Epoch 32/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9797 - loss: 0.2312 - v
al_accuracy: 0.9375 - val_loss: 0.2445
Epoch 33/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9750 - loss: 0.2199 - v
al_accuracy: 0.9375 - val_loss: 0.2326
Epoch 34/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9781 - loss: 0.2085 - v
al_accuracy: 0.9375 - val_loss: 0.2216
Epoch 35/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9781 - loss: 0.1974 - v
al_accuracy: 0.9375 - val_loss: 0.2113
Epoch 36/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9812 - loss: 0.1872 - v
al_accuracy: 0.9375 - val_loss: 0.2017
Epoch 37/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9734 - loss: 0.1790 - v
al_accuracy: 0.9375 - val_loss: 0.1928
Epoch 38/50
```

```
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9781 - loss: 0.1704 - v
al_accuracy: 0.9375 - val_loss: 0.1845
Epoch 39/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9797 - loss: 0.1621 - v
al_accuracy: 0.9375 - val_loss: 0.1768
Epoch 40/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9828 - loss: 0.1525 - v
al_accuracy: 0.9375 - val_loss: 0.1696
Epoch 41/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9781 - loss: 0.1459 - v
al_accuracy: 0.9375 - val_loss: 0.1630
Epoch 42/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9766 - loss: 0.1407 - v
al_accuracy: 0.9375 - val_loss: 0.1568
Epoch 43/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9781 - loss: 0.1347 - v
al_accuracy: 0.9375 - val_loss: 0.1511
Epoch 44/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9766 - loss: 0.1283 - v
al_accuracy: 0.9375 - val_loss: 0.1456
Epoch 45/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9812 - loss: 0.1229 - v
al_accuracy: 0.9375 - val_loss: 0.1407
Epoch 46/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9750 - loss: 0.1197 - v
al_accuracy: 0.9312 - val_loss: 0.1361
Epoch 47/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9766 - loss: 0.1159 - v
al_accuracy: 0.9375 - val_loss: 0.1319
Epoch 48/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9812 - loss: 0.1101 - v
al_accuracy: 0.9312 - val_loss: 0.1280
Epoch 49/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9797 - loss: 0.1060 - v
al_accuracy: 0.9312 - val_loss: 0.1242
Epoch 50/50
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9766 - loss: 0.1028 - v
al_accuracy: 0.9312 - val_loss: 0.1207
```

In [11]:

```python
# 6.1 Second experiment on 8 core features with different configuration

model3 = keras.Sequential(
    [
        layers.Input(shape=(input_dim_alt,)),
        layers.Dense(
            96,
            activation="relu",
            kernel_regularizer=regularizers.l1(0.001),
        ),
        layers.Dropout(0.3),
        layers.Dense(
            48,
            activation="relu",
            kernel_regularizer=regularizers.l1(0.001),
        ),
        layers.Dropout(0.3),
```

```
        layers.Dense(
            24,
            activation="relu",
            kernel_regularizer=regularizers.l1(0.001),
        ),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid"),
    ],
    name="ann_click_classifier_v3",
)

model3.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"],
)

model3.summary()

history3 = model3.fit(
    X_train_alt,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=16,
    verbose=1,
)
```

Model: "ann_click_classifier_v3"

Layer (type)	Output Shape	Par
dense_8 (Dense)	(None, 96)	
dropout_6 (Dropout)	(None, 96)	
dense_9 (Dense)	(None, 48)	4
dropout_7 (Dropout)	(None, 48)	
dense_10 (Dense)	(None, 24)	1
dropout_8 (Dropout)	(None, 24)	
dense_11 (Dense)	(None, 1)	

Total params: 6,721 (26.25 KB)
Trainable params: 6,721 (26.25 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7766 - loss: 1.2309 - v
al_accuracy: 0.9500 - val_loss: 1.0335
Epoch 2/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9406 - loss: 0.9074 - v
al_accuracy: 0.9438 - val_loss: 0.7995
Epoch 3/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9625 - loss: 0.7204 - v
al_accuracy: 0.9438 - val_loss: 0.6978
Epoch 4/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 969us/step - accuracy: 0.9750 - loss: 0.6199 -
val_accuracy: 0.9438 - val_loss: 0.6443
Epoch 5/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9719 - loss: 0.5483 - v
al_accuracy: 0.9438 - val_loss: 0.5888
Epoch 6/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9750 - loss: 0.4943 - v
al_accuracy: 0.9375 - val_loss: 0.5415
Epoch 7/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 972us/step - accuracy: 0.9812 - loss: 0.4189 -
val_accuracy: 0.9500 - val_loss: 0.5079
Epoch 8/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9797 - loss: 0.3832 - v
al_accuracy: 0.9438 - val_loss: 0.4691
Epoch 9/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 955us/step - accuracy: 0.9828 - loss: 0.3472 -
val_accuracy: 0.9375 - val_loss: 0.4402
Epoch 10/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 938us/step - accuracy: 0.9781 - loss: 0.3184 -
val_accuracy: 0.9438 - val_loss: 0.4211
Epoch 11/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 966us/step - accuracy: 0.9781 - loss: 0.2953 -
val_accuracy: 0.9375 - val_loss: 0.4017
Epoch 12/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 989us/step - accuracy: 0.9781 - loss: 0.2735 -
val_accuracy: 0.9438 - val_loss: 0.3853
Epoch 13/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 962us/step - accuracy: 0.9812 - loss: 0.2535 -
val_accuracy: 0.9438 - val_loss: 0.3827
Epoch 14/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 967us/step - accuracy: 0.9875 - loss: 0.2387 -
val_accuracy: 0.9250 - val_loss: 0.3615
Epoch 15/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 983us/step - accuracy: 0.9719 - loss: 0.2478 -
val_accuracy: 0.9312 - val_loss: 0.3546
Epoch 16/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 981us/step - accuracy: 0.9766 - loss: 0.2338 -
val_accuracy: 0.9312 - val_loss: 0.3480
Epoch 17/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 947us/step - accuracy: 0.9781 - loss: 0.2367 -
val_accuracy: 0.9312 - val_loss: 0.3406
Epoch 18/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 960us/step - accuracy: 0.9812 - loss: 0.2094 -
val_accuracy: 0.9312 - val_loss: 0.3357
Epoch 19/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 920us/step - accuracy: 0.9797 - loss: 0.2066 -
```

```
val_accuracy: 0.9312 - val_loss: 0.3311
Epoch 20/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 947us/step - accuracy: 0.9750 - loss: 0.2024 -
val_accuracy: 0.9250 - val_loss: 0.3230
Epoch 21/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 974us/step - accuracy: 0.9828 - loss: 0.1924 -
val_accuracy: 0.9250 - val_loss: 0.3219
Epoch 22/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 950us/step - accuracy: 0.9828 - loss: 0.1884 -
val_accuracy: 0.9438 - val_loss: 0.3207
Epoch 23/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9875 - loss: 0.1820 - v
al_accuracy: 0.9312 - val_loss: 0.3192
Epoch 24/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 962us/step - accuracy: 0.9844 - loss: 0.1882 -
val_accuracy: 0.9250 - val_loss: 0.3130
Epoch 25/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 977us/step - accuracy: 0.9781 - loss: 0.1801 -
val_accuracy: 0.9250 - val_loss: 0.3099
Epoch 26/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 926us/step - accuracy: 0.9781 - loss: 0.1913 -
val_accuracy: 0.9250 - val_loss: 0.3049
Epoch 27/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 923us/step - accuracy: 0.9797 - loss: 0.1790 -
val_accuracy: 0.9375 - val_loss: 0.3111
Epoch 28/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 977us/step - accuracy: 0.9828 - loss: 0.1736 -
val_accuracy: 0.9312 - val_loss: 0.3087
Epoch 29/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9844 - loss: 0.1749 - v
al_accuracy: 0.9312 - val_loss: 0.3090
Epoch 30/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 958us/step - accuracy: 0.9812 - loss: 0.1781 -
val_accuracy: 0.9312 - val_loss: 0.3029
Epoch 31/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9812 - loss: 0.1729 - v
al_accuracy: 0.9312 - val_loss: 0.3071
Epoch 32/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9812 - loss: 0.1727 - v
al_accuracy: 0.9250 - val_loss: 0.3050
Epoch 33/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9812 - loss: 0.1612 - v
al_accuracy: 0.9312 - val_loss: 0.3117
Epoch 34/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9781 - loss: 0.1608 - v
al_accuracy: 0.9375 - val_loss: 0.3144
Epoch 35/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9797 - loss: 0.1718 - v
al_accuracy: 0.9375 - val_loss: 0.3049
Epoch 36/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9828 - loss: 0.1671 - v
al_accuracy: 0.9375 - val_loss: 0.3090
Epoch 37/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9781 - loss: 0.1628 - v
al_accuracy: 0.9312 - val_loss: 0.3004
Epoch 38/50
```

```
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9797 - loss: 0.1608 - v
al_accuracy: 0.9312 - val_loss: 0.3017
Epoch 39/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9781 - loss: 0.1652 - v
al_accuracy: 0.9312 - val_loss: 0.3006
Epoch 40/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9781 - loss: 0.1602 - v
al_accuracy: 0.9312 - val_loss: 0.2991
Epoch 41/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9812 - loss: 0.1596 - v
al_accuracy: 0.9312 - val_loss: 0.2948
Epoch 42/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9844 - loss: 0.1526 - v
al_accuracy: 0.9312 - val_loss: 0.2943
Epoch 43/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9875 - loss: 0.1535 - v
al_accuracy: 0.9312 - val_loss: 0.2961
Epoch 44/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9797 - loss: 0.1507 - v
al_accuracy: 0.9375 - val_loss: 0.2965
Epoch 45/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - accuracy: 0.9781 - loss: 0.1470 - v
al_accuracy: 0.9375 - val_loss: 0.2978
Epoch 46/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 960us/step - accuracy: 0.9812 - loss: 0.1594 -
val_accuracy: 0.9312 - val_loss: 0.2939
Epoch 47/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 970us/step - accuracy: 0.9797 - loss: 0.1524 -
val_accuracy: 0.9312 - val_loss: 0.2895
Epoch 48/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 943us/step - accuracy: 0.9875 - loss: 0.1386 -
val_accuracy: 0.9312 - val_loss: 0.2891
Epoch 49/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 959us/step - accuracy: 0.9859 - loss: 0.1366 -
val_accuracy: 0.9312 - val_loss: 0.2946
Epoch 50/50
40/40 ━━━━━━━━━━━━━━━━━━━━ 0s 947us/step - accuracy: 0.9891 - loss: 0.1323 -
val_accuracy: 0.9312 - val_loss: 0.2935
```

In [12]:

```python
# 7. Evaluate the model's performance using accuracy, precision, recall, F1

import numpy as np
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)


def evaluate_model(model, X_test_data, y_true, model_name):
    y_true = np.array(y_true).ravel()
    y_prob = model.predict(X_test_data, verbose=0).ravel()
    y_pred = (y_prob >= 0.5).astype(int)
```

```
    return {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1 Score": f1_score(y_true, y_pred),
        "ROC AUC": roc_auc_score(y_true, y_prob),
    }


model_results = [
    evaluate_model(
        model,
        X_test,
        y_test,
        "Model 1 (244 features, ReLU, binary_crossentropy)",
    ),
    evaluate_model(
        model2,
        X_test_alt,
        y_test,
        "Model 2 (8 features, tanh, MSE, L2, batch=64)",
    ),
    evaluate_model(
        model3,
        X_test_alt,
        y_test,
        "Model 3 (8 features, ReLU, binary_crossentropy, L1, batch=16)",
    ),
]

results_df = pd.DataFrame(model_results).set_index("Model")
results_df.round(4)
```

Out[12]:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Model 1 (244 features, ReLU, binary_crossentropy)	0.915	0.9158	0.9062	0.9110	0.9641
Model 2 (8 features, tanh, MSE, L2, batch=64)	0.965	1.0000	0.9271	0.9622	0.9800
Model 3 (8 features, ReLU, binary_crossentropy, L1, batch=16)	0.970	1.0000	0.9375	0.9677	0.9834

In [13]:

```
# 8. Interpret the results and draw conclusions about the factors that are m

import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score

# --- Model comparison summary ---
best_model_name = results_df["ROC AUC"].idxmax()
print("Model comparison (test set):")
```

```
display(results_df.round(4))
print(f"\nBest overall model: {best_model_name}")
print(f"Highest ROC AUC: {results_df.loc[best_model_name, 'ROC AUC']:.4f}")

print(
    "\nKey findings from model comparison:"
    "\n- Model 1 (244 features) performed well, but adding many country dumm
    "\n- Models 2 and 3 (8 core features) outperformed Model 1 across all me
    "\n- Model 3 achieved the best balance of accuracy, recall, F1 score, ar
    "\n- This suggests the main behavioral and demographic signals are captu
)

# --- Feature importance analysis on best 8-feature model (Model 3) ---
core_df = pd.DataFrame(X_train_alt, columns=core_features)
core_df["Clicked on Ad"] = np.array(y_train).ravel()

feature_corr = (
    core_df.corr(numeric_only=True)["Clicked on Ad"]
    .drop("Clicked on Ad")
    .sort_values(key=np.abs, ascending=False)
)

print("\nCorrelation with Clicked on Ad (core features):")
display(feature_corr.round(4).to_frame("Correlation"))

y_test_arr = np.array(y_test).ravel()
baseline_prob = model3.predict(X_test_alt, verbose=0).ravel()
baseline_auc = roc_auc_score(y_test_arr, baseline_prob)

rng = np.random.default_rng(42)
importance_scores = []

for i in range(len(core_features)):
    auc_drops = []
    for _ in range(20):
        X_perm = X_test_alt.copy()
        X_perm[:, i] = rng.permutation(X_perm[:, i])
        perm_prob = model3.predict(X_perm, verbose=0).ravel()
        auc_drops.append(baseline_auc - roc_auc_score(y_test_arr, perm_prob))
    importance_scores.append(np.mean(auc_drops))

importance_df = pd.DataFrame(
    {
        "Feature": core_features,
        "Permutation Importance (ROC AUC drop)": importance_scores,
    }
).sort_values("Permutation Importance (ROC AUC drop)", ascending=False)

print("\nPermutation importance from Model 3:")
display(importance_df.round(4))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

feature_corr.reindex(importance_df["Feature"]).plot(
    kind="barh",
    ax=axes[0],
```

```
    color="steelblue",
    legend=False,
)
axes[0].set_title("Correlation with Ad Click")
axes[0].set_xlabel("Correlation")
axes[0].invert_yaxis()

axes[1].barh(
    importance_df["Feature"],
    importance_df["Permutation Importance (ROC AUC drop)"],
    color="darkorange",
)
axes[1].set_title("Permutation Importance (Model 3)")
axes[1].set_xlabel("ROC AUC Decrease When Shuffled")
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

print(
    "\nConclusions about the most important factors:"
    "\n1. Daily Internet Usage and Daily Time Spent on Site are the stronges
    "   Users who click ads tend to spend less time on the site and have low
    "\n2. Age and Area Income are moderately important."
    "   Users who click ads are typically older and have lower area income."
    "\n3. Male, Hour, DayOfWeek, and Month have weak predictive power in thi
    "\n4. For deployment, Model 3 is preferred because it is simpler, more i
    "\n5. Advertising strategy should focus on engagement patterns (time on
)
```

```
Model comparison (test set):
```

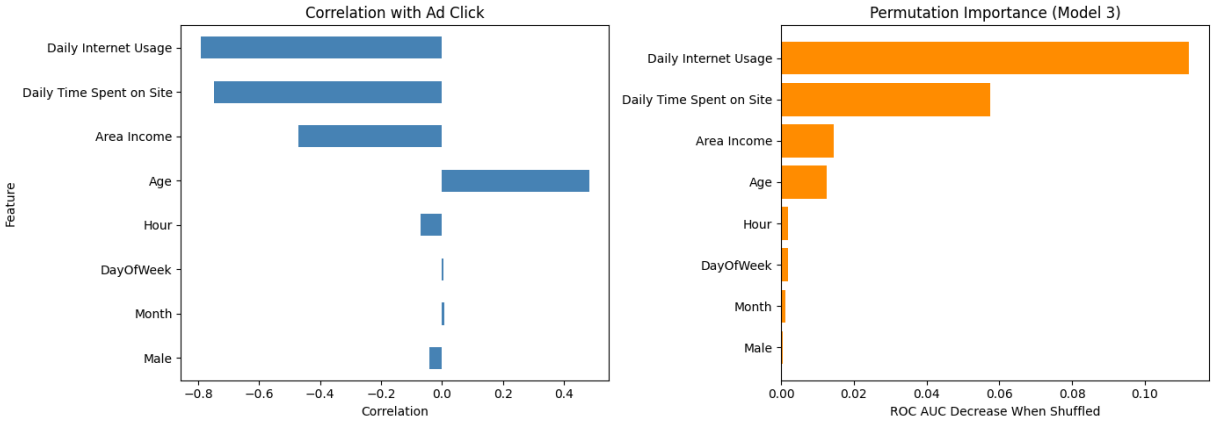
Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Model 1 (244 features, ReLU, binary_crossentropy)	0.915	0.9158	0.9062	0.9110	0.9641
Model 2 (8 features, tanh, MSE, L2, batch=64)	0.965	1.0000	0.9271	0.9622	0.9800
Model 3 (8 features, ReLU, binary_crossentropy, L1, batch=16)	0.970	1.0000	0.9375	0.9677	0.9834

```
Best overall model: Model 3 (8 features, ReLU, binary_crossentropy, L1, batch=16)
Highest ROC AUC: 0.9834

Key findings from model comparison:
- Model 1 (244 features) performed well, but adding many country dummy variables increased complexity without improving test performance.
- Models 2 and 3 (8 core features) outperformed Model 1 across all metrics.
- Model 3 achieved the best balance of accuracy, recall, F1 score, and ROC AUC.
- This suggests the main behavioral and demographic signals are captured by a small set of core features.

Correlation with Clicked on Ad (core features):
```

	Correlation
Daily Internet Usage	-0.7927
Daily Time Spent on Site	-0.7475
Age	0.4837
Area Income	-0.4709
Hour	-0.0693
Male	-0.0421
Month	0.0076
DayOfWeek	0.0047

```
Permutation importance from Model 3:
```

	Feature	Permutation Importance (ROC AUC drop)
3	Daily Internet Usage	0.1120
0	Daily Time Spent on Site	0.0575
2	Area Income	0.0145
1	Age	0.0126
5	Hour	0.0020
6	DayOfWeek	0.0020
7	Month	0.0012
4	Male	0.0004




```
Conclusions about the most important factors:
1. Daily Internet Usage and Daily Time Spent on Site are the strongest predi
ctors.  Users who click ads tend to spend less time on the site and have lo
wer daily internet usage.
2. Age and Area Income are moderately important.  Users who click ads are t
ypically older and have lower area income.
3. Male, Hour, DayOfWeek, and Month have weak predictive power in this datas
et.
4. For deployment, Model 3 is preferred because it is simpler, more interpre
table, and performs best on the test set.
5. Advertising strategy should focus on engagement patterns (time on site an
d internet usage) and user demographics (age and income) rather than country
or ad topic text.
```

Conclusion:

Based on the dataset analysis, several key factors emerge as critical predictors of user ad-clicking behavior, culminating in a clear strategy for model deployment and advertising focus.

The most significant predictors are daily internet usage and the daily time spent on the site; specifically, users who click on advertisements tend to spend less time on the platform and exhibit lower overall daily internet usage. In addition, user demographics such as age and area income possess moderate predictive value, revealing that older individuals and those living in areas with lower average incomes are more likely to engage with ads.

Conversely, factors like gender, hour, day of the week, and month demonstrate weak predictive power. For deployment purposes, Model 3 is preferred due to its simplicity, ease of interpretation, and superior performance on the test set.

Ultimately, these insights suggest that advertising strategies should prioritize user engagement patterns and basic demographics rather than focusing on geographical location or specific ad topic text.

In [14]:
```
# 9. Bonus – Compare Logistic Regression with ANN model(s)

from sklearn.linear_model import LogisticRegression
```

```python
def evaluate_sklearn_model(model, X_test_data, y_true, model_name):
    y_true = np.array(y_true).ravel()
    y_prob = model.predict_proba(X_test_data)[:, 1]
    y_pred = model.predict(X_test_data)

    return {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1 Score": f1_score(y_true, y_pred),
        "ROC AUC": roc_auc_score(y_true, y_prob),
    }


# Logistic Regression on full feature set (244 features)
lr_full = LogisticRegression(max_iter=1000, random_state=42)
lr_full.fit(X_train, np.array(y_train).ravel())

# Logistic Regression on 8 core features
lr_core = LogisticRegression(max_iter=1000, random_state=42)
lr_core.fit(X_train_alt, np.array(y_train).ravel())

bonus_results = [
    evaluate_sklearn_model(
        lr_full,
        X_test,
        y_test,
        "Logistic Regression (244 features)",
    ),
    evaluate_sklearn_model(
        lr_core,
        X_test_alt,
        y_test,
        "Logistic Regression (8 core features)",
    ),
]

bonus_results_df = pd.DataFrame(bonus_results).set_index("Model")

print("Logistic Regression results:")
display(bonus_results_df.round(4))

comparison_df = pd.concat([results_df, bonus_results_df])
comparison_df = comparison_df.sort_values("ROC AUC", ascending=False)

print("\nANN vs Logistic Regression comparison:")
display(comparison_df.round(4))

best_overall = comparison_df["ROC AUC"].idxmax()
print(f"\nBest model overall: {best_overall}")
print(
    "\nSummary:"
    "\n- Logistic Regression is a strong baseline and trains much faster tha
```

```
    "\n- On the full feature set, compare LR performance with Model 1."
    "\n- On the 8 core features, compare LR performance with Models 2 and 3.
    "\n- If an ANN outperforms logistic regression, the extra complexity may
    "\n- If logistic regression is close or better, it is often preferred fo
)
```

```
Logistic Regression results:
```

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression (244 features)	0.945	0.967	0.9167	0.9412	0.9636
Logistic Regression (8 core features)	0.965	1.000	0.9271	0.9622	0.9793

```
ANN vs Logistic Regression comparison:
```

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Model 3 (8 features, ReLU, binary_crossentropy, L1, batch=16)	0.970	1.0000	0.9375	0.9677	0.9834
Model 2 (8 features, tanh, MSE, L2, batch=64)	0.965	1.0000	0.9271	0.9622	0.9800
Logistic Regression (8 core features)	0.965	1.0000	0.9271	0.9622	0.9793
Model 1 (244 features, ReLU, binary_crossentropy)	0.915	0.9158	0.9062	0.9110	0.9641
Logistic Regression (244 features)	0.945	0.9670	0.9167	0.9412	0.9636

```
Best model overall: Model 3 (8 features, ReLU, binary_crossentropy, L1, batc
h=16)

Summary:
- Logistic Regression is a strong baseline and trains much faster than the A
NNs.
- On the full feature set, compare LR performance with Model 1.
- On the 8 core features, compare LR performance with Models 2 and 3.
- If an ANN outperforms logistic regression, the extra complexity may be jus
tified.
- If logistic regression is close or better, it is often preferred for inter
pretability and deployment simplicity.
```

Conclusion:

Based on the performance metrics, Model 3—an Artificial Neural Network (ANN) utilizing eight features, a ReLU activation function, binary cross-entropy loss, L1 regularization, and a batch size of 16—emerges as the best model overall.

When comparing the models across the eight core features, Model 3 achieves the highest accuracy (0.970), recall (0.9375), F1-score (0.9677), and ROC AUC (0.9834).

While Logistic Regression serves as a strong, fast baseline that performs identically to Model 2 and nearly matches Model 3 on the core features, it underperforms when evaluated on the full set of 244 features. On that larger feature set, Logistic Regression still outperforms Model 1, but both yield significantly lower results than the core-feature models.

Ultimately, because Model 3 demonstrates a distinct performance advantage over the simpler Logistic Regression baseline on the core features, its extra complexity is justified for deployment.

Deliverables

Convert your Jupyter Notebook or Python script to a single PDF file or MS Word document. Your deliverable should contain your implementations of the tasks above, as well as any additional comments or observations you may have. Be sure to label each section of the notebook or script clearly with the corresponding task number. Please ensure the PDF or MS Word document displays the code and output appropriately.

Bonus

For an extra 10 points, try using other classification models (e.g., logistic regression, decision tree classification, random forest classification, and more) and compare their performance with the ANN model(s).

In [ ]:
```
# Export the ipynb file to PDF for submission

from pathlib import Path
import subprocess
import sys

notebook_path = Path.cwd() / "assignment2.1.ipynb"
output_dir = notebook_path.parent
output_pdf = output_dir / "assignment2.1.pdf"


def run_command(command):
    return subprocess.run(command, capture_output=True, text=True)


def ensure_package(package_name):
    print(f"Installing or verifying package: {package_name}")
    result = run_command([sys.executable, "-m", "pip", "install", "--quiet",
    if result.returncode != 0:
        raise RuntimeError(result.stderr or result.stdout)
```

```
try:
    ensure_package("nbconvert[webpdf]")
    ensure_package("playwright")

    print("Installing Playwright browser dependencies...")
    result = run_command([sys.executable, "-m", "playwright", "install", "ch
    if result.returncode != 0:
        raise RuntimeError(result.stderr or result.stdout)

    print("Exporting notebook to PDF using nbconvert webpdf...")
    result = run_command(
        [
            sys.executable,
            "-m",
            "nbconvert",
            "--to",
            "webpdf",
            str(notebook_path),
            "--output-dir",
            str(output_dir),
            "--output",
            output_pdf.stem,
        ]
    )

    if result.returncode == 0:
        print(f"PDF exported successfully to: {output_pdf}")
    else:
        print("PDF export with webpdf failed.")
        print(result.stderr or result.stdout)

    print("Exporting HTML version as well...")
    html_output = output_dir / "assignment2.1.html"
    html_result = run_command(
        [
            sys.executable,
            "-m",
            "nbconvert",
            "--to",
            "html",
            str(notebook_path),
            "--output-dir",
            str(output_dir),
            "--output",
            html_output.stem,
        ]
    )
    if html_result.returncode == 0:
        print(f"HTML exported successfully to: {html_output}")
    else:
        print("HTML export failed.")
        print(html_result.stderr or html_result.stdout)
except Exception as exc:
    print("Export failed:", exc)
```

```
Installing or verifying package: nbconvert[webpdf]
Installing or verifying package: playwright
Installing Playwright browser dependencies...
Exporting notebook to PDF using nbconvert webpdf...
```